

LTS Pricing Calculator

ASP.NET Core 6 Razor Pages — Integration & Architecture Guide

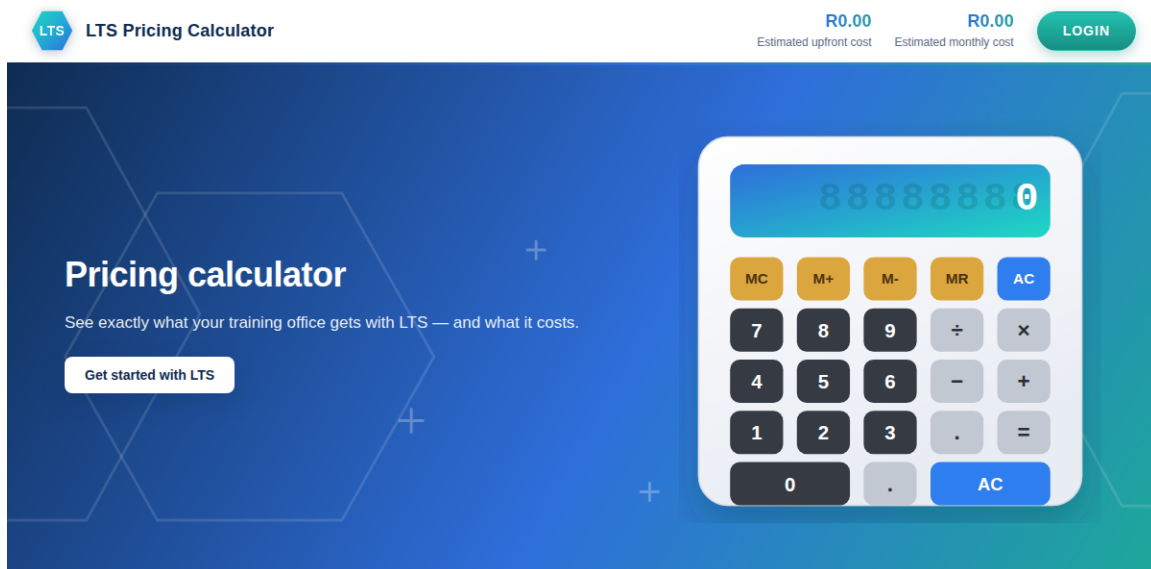
1. Overview

The LTS Pricing Calculator is a client-side pricing tool: a customer picks products, sets trainee/user counts, and sees live pricing across contract terms, an annual escalation projection, and a comparison against doing the same admin manually. There's no database and no server-side logic. Every calculation runs in the browser, in plain JavaScript.

This guide covers two things: how the code works end to end, and what to do to drop it into an existing ASP.NET Core 6 Razor Pages site.

Requires a Razor Pages project (one with a `Pages/` folder and `builder.Services.AddRazorPages()` / `app.MapRazorPages()` in `Program.cs`), not a plain MVC project (`Controllers/` + `Views/`) with no Razor Pages support added. Most ASP.NET Core 6 web app templates include both by default. If yours is MVC-only, add Razor Pages support with those two lines before following section 4.

What it looks like once integrated:



2. How the code works, end to end

The whole app is four JavaScript files, loaded in order, plus one page of HTML. Nothing is compiled or bundled; they're concatenated as-is for the Razor Pages version.

2.1 Data layer — `data.js`

A single object, `LTS_DATA`, holds everything that isn't logic: company info, VAT rate, the confirmed annual price escalation percentage, every product's name/description/pricing model, the trainee volume brackets, the contract terms, the full rate table (rate per trainee per month, keyed by contract and bracket), ready-made estimate templates, default assumptions for the "cost of NOT using LTS" comparison, and the FAQ content.

That's on purpose. Change a price, add a product, or update an FAQ, and **only this file changes**. Nothing else in the codebase hardcodes a price or product name.

```
const LTS_DATA = {
  vatRate: 0.15,
  annualEscalationPct: 6.6,
  products: [ { id: "platform", pricingModel: "tiered", ... },
    { id: "timesheet", pricingModel: "flat", flatRate: 59.24 } ],
  tiers: [ { id: "1-29", min: 1, max: 29 }, ... ],
  contracts: [ { id: "payg" }, { id: "1yr" }, { id: "2yr" }, { id: "3yr" } ],
  rates: { payg: { "1-29": 418.23, ... }, "3yr": { ... } },
  manualBaseline: { trainees: 50, adminRate: 220, ... },
  templates: [ ... ], faqs: [ ... ],
};
```

2.2 Pricing engine — `calculator.js`

Pure functions with no DOM access: everything here takes numbers or ids in and returns numbers out. That's what makes it testable on its own, and why the Excel export can reuse these same functions instead of recalculating separately, so the totals and the export can never disagree.

- **`calcLine(trainees, contractId)`** looks up the correct rate bracket for the platform product and returns the monthly/annual cost, excl. and incl. VAT.
- **`calcEstimateLine(line)`** is the dispatcher every line goes through. Tiered products (the platform) delegate to `calcLine`; flat-rate products (Time Sheet) multiply units by the product's flat rate, or return a "pricing to be confirmed" placeholder if no rate is published yet.
- **`estimatedAnnual(monthlyExclVat, fromDate)`** projects the next 12 months forward from today, blending in the confirmed annual escalation on whichever side of 1 April the projection falls.
- **`manualBaselineCost(assumptions)`** is the "cost of NOT using LTS" model: four categories (learner & programme admin, monitoring & follow-up, reporting & compliance, management time), each hours times its own rate, summed to a total "without LTS" monthly figure.
- **`calcSavingsVsPayg`** and **`calcAllContracts`** power the contract-term comparison grid on each line item.

2.3 App wiring — `app.js`

This is the only file that touches the DOM. It holds one state object (the list of estimate line items, VAT toggle, and the "cost of NOT using LTS" assumptions), plus a set of render functions that redraw the affected part of the page whenever state changes. There's no framework here, just direct `innerHTML` updates and event listeners re-attached after each render.

- **`renderProducts()`** draws the two product cards from `LTS_DATA.products`.

- **renderEstimate()**, **renderTotals()** and **renderComparison()** form the main render chain: any change to a line item re-renders the estimate list, then totals, then the comparison card, so the top-bar figures, totals card, and comparison numbers stay in sync.
- The trainee count in "cost of NOT using LTS" **auto-fills** from the sum of every estimate line's people (trainees for the platform, units for Time Sheet) until the user edits that field directly. After that, it stops auto-syncing and stays as the user's own number.
- **calcLoader** is the Azure-style loading overlay shown briefly when jumping to the estimate.

2.4 Export — export.js

Builds an Excel workbook client-side using the SheetJS (xlsx) library loaded from a CDN, re-using calculator.js's own functions so the exported numbers always match what's on screen. Also has helpers for saving/restoring an estimate to the browser's localStorage.

2.5 The Razor Page — Index.cshtml

The Razor Pages version is a straight port: the original HTML file's <body> markup is copied verbatim into the page, and the four JS files are concatenated in their original load order into one pricing-calculator.js. The page sets `Layout = null`; so it renders its own complete HTML document instead of using the host site's shared layout. More on that in section 4.

3. File map

Path	What it is
Pages/Index.cshtml	The entire page. Self-contained: its own <html>/<head>/<body>, no PageModel.
wwwroot/css/pricing-calculator.css	All styling. Byte-identical to the original site's stylesheet.
wwwroot/js/pricing-calculator.js	data.js + calculator.js + export.js + app.js, concatenated.
wwwroot/assets/*.svg	Hero illustration, background pattern, decorative hex tile.
PROGRAM_CS_NOTES.md	The one line to confirm exists in your Program.cs.
README.md	Integration steps (mirrors section 4 of this guide).

4. Integrating into your ASP.NET Core 6 project — plug and play

The page is self-contained on purpose, so this is a copy-and-check operation, not a merge. It won't collide with your project's namespace, navbar, or shared layout.

Step 1 — Copy four things into your project

```
YourProject/  
Pages/Index.cshtml <- copy from this package  
wwwroot/css/pricing-calculator.css <- copy from this package  
wwwroot/js/pricing-calculator.js <- copy from this package  
wwwroot/assets/*.svg <- copy from this package
```

About the route: Razor Pages maps a page's route from its file path, so Pages/Index.cshtml serves at /, your site's root/home page. Most default ASP.NET Core 6 templates already have their own Pages/Index.cshtml there, and copying this one over it would replace your home page with the calculator. That's rarely what you want, so in almost every real integration you should **rename the file** before copying it in. Pages/PricingCalculator.cshtml, for example, serves at /PricingCalculator instead, leaving your existing home page untouched.

Step 2 — Confirm one line in Program.cs

Almost every ASP.NET Core 6 project already has this; it's what serves everything under wwwroot/:

```
app.UseStaticFiles();
```

See PROGRAM_CS_NOTES.md in the package for a full reference Program.cs to compare against.

Step 3 — Build and run

dotnet build, then dotnet run, then open the page's route in a browser. That's the full integration: no namespace to rename, no PageModel to wire up, no blocks to merge into an existing _Layout.cshtml.

Why no layout merge is needed

Index.cshtml sets `Layout = null;` in its code block, so it renders its own complete `<html><head><body>` document instead of participating in the host site's shared `_Layout.cshtml`. That's what makes it safe to drop in. It can't be affected by the host site's navbar, Bootstrap container, or CSS bundle, and the host site's layout can't be affected by it either.

An earlier version of this port depended on three small edits merged into the host's `_Layout.cshtml` (hide the shared navbar, skip the Bootstrap container, render a Styles section) plus a namespace fix in a separate PageModel file. Both of those manual steps have been removed in this version.

5. Troubleshooting

Symptom	Likely cause / fix
Page loads but is unstyled (plain black-and-white HTML)	<code>app.UseStaticFiles();</code> is missing, or placed after <code>app.UseRouting()</code> in <code>Program.cs</code> . Move it earlier, per <code>PROGRAM_CS_NOTES.md</code> .
Browser console shows 404 for <code>pricing-calculator.css</code> or <code>.js</code>	The files aren't under <code>wwwroot/</code> in the exact folder structure shown in section 3, or the casing doesn't match (Linux hosting is case-sensitive).
Page loads, but nothing is clickable and no products show	<code>pricing-calculator.js</code> failed to load or errored before running. Check the browser console for a script error, and confirm the <code><script></code> tag path is correct.
"Export to Excel" button does nothing	The SheetJS CDN (<code>cdnjs.cloudflare.com</code>) is blocked by your network or firewall. See the external dependency note below.
The calculator replaced your site's home page	<code>Index.cshtml</code> serves at <code>/</code> by convention. Rename it per the note in Step 1 above.

6. Updating prices, products or FAQs later

Everything price/product/FAQ-related lives in one place: the `LTS_DATA` object at the top of `pricing-calculator.js` (marked "1/4: data.js" in the file's comments). Change a rate, add a product, or edit an FAQ answer there, and nothing else in the file needs to change, since every other function reads from this object rather than hardcoding values.

- **New product:** add an entry to `LTS_DATA.products` (tiered products need a rate table entry; flat-rate products just need a `flatRate`, or omit it to show "pricing to be confirmed").
- **Rate change:** edit `LTS_DATA.rates` directly, or the `flatRate` on a flat product.
- **Escalation change:** edit `LTS_DATA.annualEscalationPct`.

- **New FAQ:** add to LTS_DATA.faqs, grouped by category.

7. What's been verified, and what hasn't

- **Verified:** the CSS is a byte-for-byte copy of the original stylesheet (diff-checked). The HTML body is extracted programmatically from the original source and diff-verified, not retyped. The JavaScript bundle is syntax-checked (node --check) and functionally tested in a browser (Playwright): adding lines, switching contracts, the VAT toggle, Excel export, the comparison calculator.
- **Not verified:** an actual dotnet build/run of this exact Razor Pages structure. This guide was produced in an environment without the .NET SDK installed. The Razor syntax has been manually audited (brace balance, every @ usage accounted for, HTML tag balance checked), but do a local build before shipping to production.
- **External dependency:** the Excel export relies on the SheetJS (xlsx) library loaded from a CDN (cdnjs.cloudflare.com). If your deployment environment blocks that CDN, download the library and serve it from wwwroot instead, updating the <script> tag's src.